

团队分工上传计划

人员分工总览

成员	负责板块	角色定位	文件数	总代码量
杨	核心数据结构 (core/ 底层)	底层架构师	5 个	~6.3 KB
赵	业务逻辑层 (core/ 上层)	业务开发	5 个	~16.4 KB
罗	CLI + 测试 + 构建脚本	测试与构建工程师	7 个	~41.0 KB
林	Web 后端服务器	后端开发	1 个	~19.7 KB
陈	Web 前端 + 文档	前端与文档	6 个	~49.0 KB

注：代码量大的成员（陈、罗）文件数量少，代码量小的成员（杨）文件数量多，总体工作量均衡。

上传规范

仓库目录结构

上传到 Gitee 后，仓库目录结构必须保持与源码一致：

```
paypass-lite/
├── core/                # 核心库
│   ├── account.h
│   ├── account.cpp
│   ├── hashtable.h
│   ├── hashtable.cpp
│   ├── stack.h
│   ├── stack.cpp
│   ├── transaction.h
│   ├── transaction.cpp
│   ├── ledger_system.h
│   └── ledger_system.cpp
├── cli/
│   └── main_cli.cpp
├── web/
│   ├── main_server.cpp
│   └── static/
│       ├── index.html
│       ├── style.css
│       └── app.js
├── docs/
│   └── MANUAL.md
├── test_main.cpp
├── README.md
├── AGENTS.md
├── build.bat
├── build.sh
├── run.bat
├── run.sh
├── start_server.bat
├── test.bat
├── test.sh
└── test.ps1
```

提交信息规范

格式： [模块名] 操作： 具体内容

类型	格式示例
新增文件	[core] add: 实现哈希表插入和查找功能
修改文件	[web] update: 完善转账接口错误处理
修复Bug	[cli] fix: 修复中文菜单显示乱码
完善文档	[docs] update: 补充API使用示例

分支策略（可选）

main

← 最终合并

├─ dev-yang

├─ dev-zhao

├─ dev-luo

├─ dev-lin

└─ dev-chen

← 杨：数据结构开发分支

← 赵：业务逻辑开发分支

← 罗：CLI与测试开发分支

← 林：Web后端开发分支

← 陈：前端与文档开发分支

每人每天在自己的分支上提交，第10天合并到 main。

详细日程表（10天）

第1天 — 项目初始化

目标：搭建项目骨架，确定接口规范

成员	文件路径	提交信息	说明
杨	core/hashtable.h	[core] add: 哈希表头文件，定义HashTable结构体和djb2哈希函数	哈希表数据结构定义，链地址法，容量101
赵	core/account.h	[core] add: 账户类头文件，定义账户结构、余额和交易历史接口	Account类声明，包含交易链表头指针
罗	build.bat	[build] add: Windows构建脚本，编译所有二进制文件	g++编译命令，-std=c++14 -Wall -Wextra
罗	build.sh	[build] add: Linux构建脚本，编译所有二进制文件	同上，Shell版本
林	web/main_server.cpp	[web] add: HTTP服务器框架，实现socket初始化和请求解析骨架	前50%代码：Winsock2/POSIX socket、HTTP/1.1解析器
陈	README.md	[docs] add: 项目说明文档，包含项目概述和快速开始	项目介绍、目录结构、构建运行说明

第2天

目标：实现核心数据结构，搭建前后端基础

成员	文件路径	提交信息	说明
杨	core/hashtable.cpp	[core] add: 哈希表实现, 插入/查找/删除/遍历操作	链地址法实现, djb2哈希
赵	core/account.cpp	[core] add: 账户类实现, 构造函数/余额操作/交易历史管理	账户创建、存款取款、交易链表操作
罗	test_main.cpp	[test] add: 哈希表和账户的单元测试用例	前40%测试代码: 测试HashTable和Account
林	web/main_server.cpp	[web] update: 实现REST API路由, 添加账户创建和查询接口	后50%代码: 路由注册、JSON序列化
陈	docs/MANUAL.md	[docs] add: 用户手册, 包含项目介绍和CLI使用说明	前半部分: 项目概述、CLI操作指南

第3天

目标: 实现交易系统和双向链表

成员	文件路径	提交信息	说明
杨	core/transaction.h	[core] add: 交易节点和双向链表头文件	Transaction结构体、DoublyLinkedList类声明
赵	core/transaction.cpp	[core] add: 交易和双向链表实现, 添加/删除/遍历操作	链表节点管理、交易记录操作
罗	test_main.cpp	[test] update: 添加交易和链表的单元测试用例	中间30%测试代码: 测试Transaction和链表
林	web/static/index.html	[web] add: 前端HTML页面, 侧边栏+内容面板布局	单页应用结构, 导航菜单
陈	docs/MANUAL.md	[docs] update: 补充API参考和常见问题章节	后半部分: API接口文档、FAQ

第4天

目标: 实现撤销栈和外观类, 完善CLI和前端

成员	文件路径	提交信息	说明
杨	core/stack.h	[core] add: UndoStack头文件，固定大小数组栈(容量1000)	栈结构定义，撤销操作接口
赵	core/ledger_system.h	[core] add: LedgerSystem外观类头文件，封装所有业务逻辑接口	外观模式，统一账户/交易/撤销操作
罗	cli/main_cli.cpp	[cli] add: CLI菜单框架，实现菜单显示和用户输入循环	前50%代码：中英双语菜单、输入处理
林	web/static/style.css	[web] add: 前端样式，支付宝蓝主题(#1677FF)响应式设计	全局样式、布局、组件样式
陈	AGENTS.md	[docs] add: 工作区规则，开发规范和构建命令说明	项目规则、开发 workflow

第5天

目标：完成核心功能实现

成员	文件路径	提交信息	说明
杨	core/stack.cpp	[core] add: UndoStack实现，push/pop/peek/撤销操作	数组栈操作实现
赵	core/ledger_system.cpp	[core] add: LedgerSystem外观类实现，账户管理/存款取款/转账/撤销	全部业务逻辑：createAccount、deposit、withdraw、transfer、undo
罗	cli/main_cli.cpp	[cli] update: 实现CLI全部功能，调用LedgerSystem各操作	后50%代码：9个菜单功能实现
林	web/static/app.js	[web] add: 前端逻辑，页面初始化和账户列表加载	前50%代码：Fetch API封装、页面初始化
陈	休息 / 代码审查	审查各模块接口一致性	确保前端调用的API与后端一致

第6天

目标：完成前后端联调，完善运行脚本

成员	文件路径	提交信息	说明
杨	代码审查	[core] review: 审查数据结构接口使用正确性	确保赵的ledger_system正确调用数据结构
赵	core/ledger_system.cpp	[core] update: 完善转账和交易历史查询功能	补充转账原子性保证、交易历史遍历
罗	run.bat	[build] add: Windows启动脚本，编译+启动服务器+打开浏览器	一键启动
罗	run.sh	[build] add: Linux启动脚本，编译+启动服务器+打开浏览器	一键启动
林	web/static/app.js	[web] update: 实现转账操作、交易历史展示、Toast通知	后50%代码：表单提交、历史列表渲染
陈	休息 / 代码审查	审查前端与API接口匹配	确保所有API调用正确

第7天

目标：完善测试和部署脚本

成员	文件路径	提交信息	说明
杨	start_server.bat	[build] add: Windows服务器启动脚本(仅启动，不打开浏览器)	简洁启动
赵	集成调试	[core] fix: 修复业务逻辑集成问题	修复联调中发现的问题
罗	test.bat	[test] add: Windows API集成测试脚本(curl测试所有接口)	测试所有REST API
罗	test.sh	[test] add: Linux API集成测试脚本(curl测试所有接口)	同上，Shell版本
罗	test.ps1	[test] add: PowerShell API集成测试脚本	同上，PowerShell版本
林	前后端联调修复	[web] fix: 修复服务器与前端联调问题	修复CORS、JSON格式等问题
陈	文档修复	[docs] fix: 修复文档与代码不一致的地方	确保文档准确

第8天 — 边界情况修复

目标：处理各种边界情况和错误处理

成员	文件路径	提交信息	说明
杨	core/hashtable.cpp	[core] fix: 修复哈希表边界情况，空表查找和重复键处理	边界条件处理
赵	core/ledger_system.cpp	[core] fix: 修复业务逻辑边界情况，余额不足和账户不存在错误处理	错误码完善
罗	cli/main_cli.cpp	[cli] fix: 完善CLI错误提示，中英双语错误信息	用户友好提示
林	web/static/app.js	[web] fix: 完善前端表单验证和错误状态显示	输入校验、错误提示
陈	docs/MANUAL.md	[docs] update: 补充常见错误处理和注意事项	完善用户手册

第9天 — 最终完善

目标: 代码质量提升，确保所有测试通过

成员	文件路径	提交信息	说明
杨	core/ 各文件	[core] refactor: 代码审查和注释完善，确保无内存泄漏	最终审查
赵	core/ 各文件	[core] refactor: 代码审查和注释完善，确保业务逻辑正确	最终审查
罗	test_main.cpp	[test] update: 补充边界测试用例，确保ALL TESTS PASSED	全面测试
林	web/ 各文件	[web] refactor: 最终打磨，响应式适配和交互优化	前端优化
陈	README.md + docs/MANUAL.md	[docs] update: 最终文档完善，确保完整性和准确性	文档终稿

第10天 — 收尾合并

目标: 合并代码，项目交付

成员	操作	提交信息	说明
杨	合并 dev-yang → main	[merge] 合并数据结构模块到主分支	PR合并
赵	合并 dev-zhao → main	[merge] 合并业务逻辑模块到主分支	PR合并
罗	合并 dev-luo → main	[merge] 合并CLI和测试模块到主分支	PR合并
林	合并 dev-lin → main	[merge] 合并Web后端模块到主分支	PR合并
陈	合并 dev-chen → main	[merge] 合并前端和文档模块到主分支	PR合并
全体	最终验证	[project] 项目v1.0发布，所有功能完成	最终提交

提交节奏建议

时间段	操作
上午 9:00-10:00	每人上传当天文件，写提交信息
下午 14:00-15:00	查看队友提交，了解整体进度
晚上 20:00前	如有修改，补充提交

Gitee 操作流程

```
# 第1步: 克隆仓库 (仅第1天执行)
git clone https://gitee.com/你们的仓库地址/paypass-lite.git
cd paypass-lite

# 第2步: 创建自己的分支 (仅第1天执行)
git checkout -b dev-杨    # 杨执行
git checkout -b dev-赵    # 赵执行
# ...以此类推

# 第3步: 每天上传文件
# 将文件复制到对应目录后执行:
git add core/hashtable.h    # 添加具体文件
git commit -m "[core] add: 哈希表头文件, 定义HashTable结构体和djb2哈希函数"
git push origin dev-杨      # 推送到自己的分支

# 第10天: 合并到主分支
git checkout main
git merge dev-杨
git push origin main
```